

Assignment 2: The Socket Exchange

Experiment Report

Drushya Salunke (2024CS10565)

Sohana (2024CS10251)

September 9, 2026

Evidence file

Contents

1	Implementation Decisions	3
1.1	Concurrency / I/O Model	3
1.2	Message Framing	3
1.3	Client Role Enforcement	3
1.4	Order Book & Matching	4
1.5	Connection Handling	4
1.6	Other Design Decisions	4
2	Experiment 1 — Listening and Connected Sockets	5
2.1	Investigation Approach	5
2.2	Answer	6
3	Experiment 2 — Observing TCP Connection States	7
3.1	Investigation Approach	7
3.2	Answer	8
4	Experiment 3 — TCP as a Byte Stream	9
4.1	Investigation Approach	9
4.2	Answer	10
5	Experiment 4 — One Client Should Not Stall the Others	11
5.1	Investigation Approach	11
5.2	Answer	12
6	Experiment 5 — Multiple Clients and I/O Multiplexing (Optional)	13
6.1	Investigation Approach	17
6.2	Answer	17
7	Experiment 6 — FIN vs. RST: Orderly and Abrupt Connection Termination	18
7.1	Investigation Approach	18
7.2	Answer	19

8	Experiment 7 — Backpressure and the Slow Receiver	20
8.1	Investigation Approach	22
8.2	Answer	22
9	Experiment 8 — Unexpected Client Disconnection	23
9.1	Investigation Approach	23
9.2	Answer	24
10	Bonus — Connection Scalability and I/O Design	25
10.1	Client-Generation Program	26
10.2	Resource Measurement Table	26
10.3	Analysis	27

1 Implementation Decisions

1.1 Concurrency / I/O Model

We used a **thread-per-connection** model. The server's main loop just calls `accept()` in a loop; for every accepted connection it spawns a daemon `threading.Thread` running `handle_client()` and immediately goes back to `accept()`. Each client thread then blocks independently on `recv()` inside `LineReader.read_line()`.

We picked this over `select()/poll()/kqueue()` mainly for simplicity: each thread is just an ordinary blocking loop (read a line, act on it, block again) instead of a state machine driven by readiness events. Python threads release the GIL around blocking calls like `socket.recv()`, so one thread waiting on a client doesn't stop the others from running. The obvious cost is that this doesn't scale as well as an event-driven design once connection counts get large, since every thread carries its own stack – which is exactly what the Bonus experiment ends up showing.

All shared exchange state – logged-in usernames, the username-to-socket map, the order book, and per-instrument subscriber sets – is protected by a single global `threading.Lock` (`state_lock`). Order acceptance and matching (`try_match`) happen while holding this lock, so two orders arriving on different threads at the same instant can't be matched inconsistently or given duplicate IDs.

1.2 Message Framing

Framing is handled by a shared `LineReader` class (`common.py`), used identically by the server and both clients. Each socket gets its own `LineReader` with a private byte buffer. `read_line()` keeps calling `recv()` and appending to the buffer until a `\n` shows up, then splits on the first newline, returns everything before it as one message, and keeps the rest buffered for next time. This covers all three framing cases from Section 2.9: a message split across multiple `recv()` calls just accumulates until the newline appears; multiple messages in one `recv()` only release the first, with the rest sitting ready for the next call without a further `recv()`; and since splitting happens on raw bytes rather than `recv()` boundaries, a message cut at an arbitrary byte offset is handled the same way as any other partial read. `send_line()` mirrors this on the write side, looping on `sock.send()` with a `memoryview` so a partial write just resumes from where it left off.

1.3 Client Role Enforcement

A connection's role isn't declared up front – it's inferred from the first line the client sends. If it's `LOGIN <username>`, the connection becomes a Trader (username checked for uniqueness, socket stored in `trader_conns` for later notifications) and control passes to `handle_trader()`. If it's `SUBSCRIBE <instrument>`, it becomes a Market-Data client and goes to `handle_market_data_after_first()`. Anything else gets `ERROR first message must be LOGIN or SUBSCRIBE` and the connection is closed.

Once a role is fixed, each handler only accepts the commands allowed for that role: `handle_trader()` takes `BUY`, `SELL`, `CANCEL`, `QUIT` and rejects anything else with `ERROR command not allowed for trader`; `handle_market_data_after_first()` takes `SUBSCRIBE`, `UNSUBSCRIBE`, `QUIT` and rejects the rest similarly. This matches the capability table in Section 2.8.

1.4 Order Book & Matching

The order book is a flat list of order dicts (id, username, side, instrument, remaining quantity, price). On a new BUY/SELL, the server assigns the next id from a shared `itertools.count(1)` counter (advanced under `state.lock` so ids are never duplicated across threads), replies `ORDER_ACCEPTED <id>`, appends the order, and calls `try_match()` before releasing the lock.

`try_match()` scans the book for a resting order with the same instrument, the opposite side, and the exact same price – the three conditions from Section 2.6. On a match, the traded quantity is the smaller of the two remaining quantities; both orders' `qty` fields are decremented, `BOUGHT/SOLD` go to the two owning traders, and a `TRADE` update is broadcast to subscribed Market-Data clients. The scan keeps going (so one incoming order can fill against several resting orders) until the incoming order is fully filled or nothing matches; fully-filled orders get dropped from `order_book` at the end, and any leftover quantity just stays in the book.

1.5 Connection Handling

Each client's handler runs inside a `try/finally`, so regardless of how it exits – graceful `QUIT`, the peer closing the connection, or an unexpected error – the socket gets closed and the client removed from whatever shared structure it was in (`usernames/trader_conns` for a trader, `subscribers` for market-data), all under `state.lock`. Since every client is on its own thread, a failure in one thread can't stop the main `accept()` loop or any other client's thread, satisfying Section 4.4's requirement.

One decision worth flagging for the experiments: inside `read_line()`, a `ConnectionResetError` (peer sent an RST) is caught and treated exactly like a clean `recv()` returning `b""` (peer sent a FIN) – both just return `None` and the handler exits as if the client disconnected normally. So at the application level, our server can't actually tell an orderly disconnect from an abrupt one; that distinction (which Experiment 6 asks about) has to be observed at the network level with `tcpdump`, not from our own logs.

1.6 Other Design Decisions

Order ids come from one global counter, guaranteeing uniqueness for the server's lifetime. Username uniqueness is checked against the shared `usernames` set at `LOGIN` time. Both clients run a background listener thread reusing the *same* `LineReader` instance created at connection time, so any bytes already buffered while waiting for the `LOGIN/SUBSCRIBE` response aren't lost or double-read once the background thread takes over. Argument parsing follows the `host port [args]` convention the launcher scripts expect, so `run-server/run-trader/ run-market-data` can `exec` these scripts directly with whatever arguments the experiment harness passes in.

2 Experiment 1 — Listening and Connected Sockets

EXPERIMENT - 1

```
FreeBSD Handbook: https://www.FreeBSD.org/handbook/
FreeBSD FAQ: https://www.FreeBSD.org/faq/
Questions List: https://www.FreeBSD.org/lists/questions/
FreeBSD Forums: https://forums.FreeBSD.org/

Documents installed with the system are in the /usr/local/share/doc/freebsd/
directory, or can be installed later with: pkg install en-freebsd-doc
For other languages, replace "en" with a language code like de or fr.

Show the version of FreeBSD installed: freebsd-version ; uname -a
Please include that output and any error messages when posting questions.
Introduction to manual pages: man man
FreeBSD directory layout: man hier

To change this login announcement, see motd(5).
root@freebsd_a2:~ # cd socket-exchange-a2
root@freebsd_a2:~/socket-exchange-a2 # git pull
remote: Enumerating objects: 7, done.
remote: Counting objects: 180N (77%), done.
remote: Compressing objects: 180N (12%), done.
remote: Total 4 (delta 3), reused 4 (delta 3), pack-reused 0 (from 0)
Unpacking objects: 180N (4%), 943 bytes | 281.8K B/s, done.
From https://github.com/dm8tbr/socket-exchange-a2
160688a..c083782 main -> origin/main
Updating 160688a..c083782
Fast-forward
 src/common.py | 12 ++++++
 1 file changed, 9 insertions(+), 3 deletions(-)
root@freebsd_a2:~/socket-exchange-a2 # python3 experiment.py 1
root@freebsd_a2:~/socket-exchange-a2 # kill -f run-server
root@freebsd_a2:~/socket-exchange-a2 # kill -f run-trader
root@freebsd_a2:~/socket-exchange-a2 # kill -f run-market-data
root@freebsd_a2:~/socket-exchange-a2 # kill -f experiment.py
root@freebsd_a2:~/socket-exchange-a2 # python3 experiment.py 1
== Experiment 1: Listening and Connected Sockets ==

Started Exchange Server (PID 1444).
Exchange Server is listening.

Experiment client: connected from 127.0.0.1:49880

The client connection is now established and idle.
Investigate the current system state.

Press Ctrl-C when you are finished.
Exchange Server listening on 127.0.0.1:5000
Killed
root@freebsd_a2:~/socket-exchange-a2 # python3 experiment.py 1
== Experiment 1: Listening and Connected Sockets ==

Started Exchange Server (PID 1519).
Exchange Server listening on 127.0.0.1:5000
Exchange Server is listening.

Client connected: ('127.0.0.1', 47343)
Experiment client: connected from 127.0.0.1:55765

Client connected: ('127.0.0.1', 55765)
The client connection is now established and idle.
Investigate the current system state.

Press Ctrl-C when you are finished.
█

Last login: Sun Sep 6 15:56:08 on tty000
dm8tbr@freebsd-a2:~$ ssh root@192.168.64.2
root@192.168.64.2:~$ ssh root@192.168.64.2
root@192.168.64.2:~$ sockstat -4
netstat -an | grep 5000
(192.168.64.2) Password for root@freebsd_a2:
Last login: Sun Sep 6 15:56:15 2020 from 192.168.64.1
FreeBSD 14.4-RELEASE (GENERIC) releng/14.4-c275675-a56f852d145

Welcome to FreeBSD!

Release Notes, Errata: https://www.FreeBSD.org/releases/
Security Advisories: https://www.FreeBSD.org/security/
FreeBSD Handbook: https://www.FreeBSD.org/handbook/
FreeBSD FAQ: https://www.FreeBSD.org/faq/
Questions List: https://www.FreeBSD.org/lists/questions/
FreeBSD Forums: https://forums.FreeBSD.org/

Documents installed with the system are in the /usr/local/share/doc/freebsd/
directory, or can be installed later with: pkg install en-freebsd-doc
For other languages, replace "en" with a language code like de or fr.

Show the version of FreeBSD installed: freebsd-version ; uname -a
Please include that output and any error messages when posting questions.
Introduction to manual pages: man man
FreeBSD directory layout: man hier

To change this login announcement, see motd(5).
root@freebsd_a2:~ # sockstat -4
USER COMMAND PID FD PROTO LOCAL ADDRESS FOREIGN ADDRESS
root sshd-sssl 1524 7 tcp4 192.168.64.2:22 192.168.64.1:57949
root sshd-sssl 1521 7 tcp4 192.168.64.2:22 192.168.64.1:57949
root python3.12 1519 3 tcp4 127.0.0.1:5000 *:*
root python3.12 1519 5 tcp4 127.0.0.1:5000 127.0.0.1:55765
root python3.12 1518 3 tcp4 127.0.0.1:55765 127.0.0.1:5000
root sshd-sssl 1420 7 tcp4 192.168.64.2:22 192.168.64.1:57933
root sshd-sssl 1422 7 tcp4 192.168.64.2:22 192.168.64.1:57933
root sshd *12 *:*
root syslogd 1133 7 udp4 *:* *:*
root@freebsd_a2:~ # netstat -an | grep 5000
tcp4 0 0 127.0.0.1:5000 127.0.0.1:55765 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 127.0.0.1:5000 ESTABLISHED
tcp4 0 0 127.0.0.1:5000 *:* LISTEN
ffff800c3e5000 stream 0 0 0 0
root@freebsd_a2:~ # █
```

2.1 Investigation Approach

We ran `experiment.py 1` on our FreeBSD VM over SSH. It starts the server (PID 1519, listening on 127.0.0.1:5000) and connects one experiment client (127.0.0.1:55765), leaving the connection idle so we could inspect it. We used `sockstat -4` and `netstat -an` to list every TCP socket belonging to the server process.

```
$ python3 experiment.py 1
$ sockstat -4
$ netstat -an | grep 5000
```

See *Evidence File – Experiment 1*.

2.2 Answer

Question: After the client has connected, identify the TCP sockets associated with the Exchange Server. How does the listening socket differ from the socket representing the connection to the client?

`sockstat -4` shows two entries under the same server process (`python3.12`, PID 1519):

```
USER COMMAND PID FD PROTO LOCAL ADDRESS FOREIGN ADDRESS
root python3.12 1519 3 tcp4 127.0.0.1:5000 *.*
root python3.12 1519 5 tcp4 127.0.0.1:5000 127.0.0.1:55765
```

and `netstat` confirms the states:

```
tcp4 0 0 127.0.0.1.5000 127.0.0.1.55765 ESTABLISHED
tcp4 0 0 127.0.0.1.55765 127.0.0.1.5000 ESTABLISHED
tcp4 0 0 127.0.0.1.5000 *.* LISTEN
```

FD 3 is the server's original **listening socket**, created once at startup by `socket()/bind()/listen()`. Its local address is `127.0.0.1:5000`, foreign address is the wildcard `*.*` – a listening socket isn't tied to any peer, it just waits for SYNs – and its state is `LISTEN`.

FD 5 on the same PID is a different socket, created the moment `accept()` returned after the client connected. It has both a specific local address and a specific foreign address (`127.0.0.1:55765`), and its state is `ESTABLISHED`. This is the socket actually used to `send()/recv()` data with this client; the listening socket never exchanges data itself, it just stays in `LISTEN` and hands off a new fd like this one for every new connection. (The remaining `netstat` line, local port 55765, is just the client's own end of the connection, owned by the separate client process, PID 1518.)

3 Experiment 2 — Observing TCP Connection States

```
root@freebsd-a2:~ # cd socket-exchange-a2/
root@freebsd-a2:~/socket-exchange-a2 # for i in $(seq 1 25); do echo "--- $(date +%T) ---"; netstat -an | grep 5000; sleep 1; done
--- 02:58:52 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:58:53 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:58:54 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:58:55 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:58:56 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:58:57 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:58:58 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:58:59 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:00 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:01 ---
tcp4      0      0 127.0.0.1.5000      127.0.0.1.41648    ESTABLISHED
tcp4      0      0 127.0.0.1.41648    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:02 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:03 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:04 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:05 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:06 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:07 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:08 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:09 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:10 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:11 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:12 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:13 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:14 ---
tcp4      0      0 127.0.0.1.5000      *.*                 LISTEN
--- 02:59:15 ---
--- 02:59:16 ---
--- 02:59:17 ---
```

3.1 Investigation Approach

We ran `experiment.py 2` to establish a Trader Client connection, and in a second terminal polled the connection's TCP state once per second for 25 seconds to sample it across establishment, idle, and termination.

```
$ python3 experiment.py 2
$ for i in $(seq 1 25); do
```

```
echo "--- $(date +%T) ---"; netstat -an | grep 5000; sleep 1;
done
```

See Evidence File – Experiment 2.

3.2 Answer

Question: How does the TCP state of the client-server connection change during its lifetime, and what events cause the observed state changes?

From 02:58:52 through 02:58:59 the loop consistently showed three lines for port 5000:

```
tcp4 0 0 127.0.0.1.5000 127.0.0.1.41648 ESTABLISHED
tcp4 0 0 127.0.0.1.41648 127.0.0.1.5000 ESTABLISHED
tcp4 0 0 127.0.0.1.5000 *.* LISTEN
```

– the connection sat `ESTABLISHED` on both ends while idle, alongside the server’s permanent `LISTEN` socket. From 02:59:00 onward, both `ESTABLISHED` lines disappeared, leaving only `LISTEN`: the connection was fully torn down while the server kept running.

This matches the general TCP lifecycle: `CLOSED` → `SYN_SENT`/`SYN_RCVD` during the handshake → `ESTABLISHED` once both sides exchange `SYN/ACKs` (what we saw from 02:58:52–02:58:59) →, once the client sends `QUIT` and closes, a `FIN/ACK` exchange moving the client through `FIN_WAIT_1` → `FIN_WAIT_2` → `TIME_WAIT` and the server through `CLOSE_WAIT` → `LAST_ACK` → `CLOSED`.

One limitation: since we only sampled once per second, the intermediate closing states (`FIN_WAIT_*`, `CLOSE_WAIT`, `TIME_WAIT`) aren’t directly visible – the connection went from fully `ESTABLISHED` to fully absent between two consecutive samples, meaning the whole close sequence finished in under a second on loopback. That tracks: loopback round-trips are essentially instant, so the `FIN/ACK` exchange happened too fast for a 1 Hz poll to catch. `tcpdump -i lo0 port 5000`, timestamping each packet individually, would be needed to see those states directly.

4 Experiment 3 — TCP as a Byte Stream

```
-- ssh root@192.168.64.2                                -- ssh root@192.168.64.2                                ...                                -- ssh root@192.168.64.2

Last login: Mon Sep  7 14:25:20 on ttys004
dru@Orushyas-MacBook-Air-2 ~ % ssh root@192.168.64.2

[[root@192.168.64.2] Password for root@freebsd_a2:
Last login: Mon Sep  7 14:25:32 2026 from 192.168.64.1
FreeBSD 14.4-RELEASE (GENERIC) releng/14.4-n273675-a456f852d145

Welcome to FreeBSD!

Release Notes, Errata: https://www.FreeBSD.org/releases/
Security Advisories: https://www.FreeBSD.org/security/
FreeBSD Handbook:    https://www.FreeBSD.org/handbook/
FreeBSD FAQ:         https://www.FreeBSD.org/faq/
Questions List:      https://www.FreeBSD.org/lists/questions/
FreeBSD Forums:      https://forums.FreeBSD.org/

Documents installed with the system are in the /usr/local/share/doc/freebsd/
directory, or can be installed later with:  pkg install en-freebsd-doc
For other languages, replace "en" with a language code like de or fr.

Show the version of FreeBSD installed:  freebsd-version ; uname -a
Please include that output and any error messages when posting questions.
Introduction to manual pages:  man man
FreeBSD directory layout:     man hier

To change this login announcement, see motd(5).
root@freebsd_a2:~ # pkill -f run-server
root@freebsd_a2:~ # cd ~/socket-exchange-a2
root@freebsd_a2:~/socket-exchange-a2 # python3 experiment.py 3
=== Experiment 3: TCP as a Byte Stream ===

Started Exchange Server (PID 1451).
Exchange Server listening on 127.0.0.1:5000
Exchange Server is listening.

Experiment client: connected from 127.0.0.1:19864

Sending one application-level message using several separate TCP writes...

Sent 6 bytes.
Client connected: ('127.0.0.1', 19591)
Client connected: ('127.0.0.1', 19864)
Sent 10 bytes.
Sent 7 bytes.
Sent 1 bytes.

Investigate what the Exchange Server received.
Press Ctrl-C when you are finished.
^C
Experiment finished.
Stopping Exchange Server...
root@freebsd_a2:~/socket-exchange-a2 # █
```

```
-- ssh root@192.168.64.2                                -- ssh root@192.168.64.2                                -- ssh root@192.168.64.2

Last login: Mon Sep  7 14:27:49 on ttys005
dru@Orushyas-MacBook-Air-2 ~ % ssh root@192.168.64.2

[[root@192.168.64.2] Password for root@freebsd_a2:
Last login: Mon Sep  7 14:28:07 2026 from 192.168.64.1
FreeBSD 14.4-RELEASE (GENERIC) releng/14.4-n273675-a456f852d145

Welcome to FreeBSD!

Release Notes, Errata: https://www.FreeBSD.org/releases/
Security Advisories: https://www.FreeBSD.org/security/
FreeBSD Handbook:    https://www.FreeBSD.org/handbook/
FreeBSD FAQ:         https://www.FreeBSD.org/faq/
Questions List:      https://www.FreeBSD.org/lists/questions/
FreeBSD Forums:      https://forums.FreeBSD.org/

Documents installed with the system are in the /usr/local/share/doc/freebsd/
directory, or can be installed later with:  pkg install en-freebsd-doc
For other languages, replace "en" with a language code like de or fr.

Show the version of FreeBSD installed:  freebsd-version ; uname -a
Please include that output and any error messages when posting questions.
Introduction to manual pages:  man man
FreeBSD directory layout:     man hier

To change this login announcement, see motd(5).
root@freebsd_a2:~ # tcpdump -i lo0 -n port 5000
tcpdump: verbose output suppressed, use -v|-vv... for full protocol decode
listening on lo0, link-type NUL (BSD loopback), snapshot length 262144 bytes
14:28:43.200210 IP 127.0.0.1.50005 > 127.0.0.1.50000: Flags [S], seq 1207515873, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3270173576 ecr 0], length 0
14:28:43.200255 IP 127.0.0.1.50000 > 127.0.0.1.50005: Flags [R,], seq 0, ack 1207515874, win 0, length 0
14:28:43.207470 IP 127.0.0.1.19591 > 127.0.0.1.50000: Flags [S], seq 207568200, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 878476017 ecr 0], length 0
14:28:43.207554 IP 127.0.0.1.50000 > 127.0.0.1.19591: Flags [S,], seq 2686746711, ack 1, win 1277, options [mss 16344,nop,wscale 6,sackOK,TS val 3154023772 ecr 878476017], length 0
14:28:43.207649 IP 127.0.0.1.19591 > 127.0.0.1.50000: Flags [S,], seq 1, win 1277, options [nop,nop,TS val 878476017 ecr 3154023772], length 0
14:28:43.207696 IP 127.0.0.1.19591 > 127.0.0.1.50000: Flags [R,], seq 1, ack 1, win 0, options [nop,nop,TS val 878476017 ecr 3154023772], length 0
14:28:43.207853 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [S], seq 908587009, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 1119344837 ecr 0], length 0
14:28:43.207859 IP 127.0.0.1.50000 > 127.0.0.1.19864: Flags [S,], seq 2436346726, ack 908587009, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 366608324 ecr 1119344837], length 0
14:28:43.207866 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [S,], seq 1, win 1277, options [nop,nop,TS val 1119344837 ecr 366608324], length 0
14:28:43.208165 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [P,], seq 1:7, ack 1, win 1277, options [nop,nop,TS val 1119344837 ecr 366608324], length 6
14:28:43.208181 IP 127.0.0.1.50000 > 127.0.0.1.19864: Flags [S,], seq 7, win 1277, options [nop,nop,TS val 366608366 ecr 1119344837], length 0
14:28:43.208185 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [P,], seq 7:17, ack 1, win 1277, options [nop,nop,TS val 1119345099 ecr 366608366], length 10
14:28:43.215525 IP 127.0.0.1.50000 > 127.0.0.1.19864: Flags [S,], seq 17, win 1277, options [nop,nop,TS val 366608588 ecr 1119345099], length 0
14:28:43.240770 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [P,], seq 17:24, ack 1, win 1277, options [nop,nop,TS val 1119345270 ecr 366608588], length 7
14:28:43.278995 IP 127.0.0.1.50000 > 127.0.0.1.19864: Flags [S,], seq 24, win 1277, options [nop,nop,TS val 366608792 ecr 1119345270], length 0
14:28:43.251843 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [P,], seq 24:25, ack 1, win 1277, options [nop,nop,TS val 1119345480 ecr 366608792], length 1
14:28:43.251359 IP 127.0.0.1.50000 > 127.0.0.1.19864: Flags [P,], seq 1:4, ack 25, win 1277, options [nop,nop,TS val 366608967 ecr 1119345480], length 3
14:28:44.022296 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [S,], seq 4, win 1277, options [nop,nop,TS val 1119345552 ecr 366608967], length 0
14:31:30.933144 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [F,], seq 26, ack 4, win 1277, options [nop,nop,TS val 1119512459 ecr 366608967], length 0
14:31:30.932779 IP 127.0.0.1.50000 > 127.0.0.1.19864: Flags [S,], seq 26, win 1277, options [nop,nop,TS val 3666775946 ecr 1119512459], length 0
14:31:30.931624 IP 127.0.0.1.50000 > 127.0.0.1.19864: Flags [F,], seq 4, ack 26, win 1277, options [nop,nop,TS val 3666775946 ecr 1119512459], length 0
14:31:30.931440 IP 127.0.0.1.19864 > 127.0.0.1.50000: Flags [S,], seq 5, win 1277, options [nop,nop,TS val 1119512459 ecr 3666775946], length 0
█
```

4.1 Investigation Approach

experiment.py 3 starts the server, connects a client, and sends a single 24-byte message as four separate socket writes rather than one – confirmed by the client’s log:

```
$ python3 experiment.py 3
Sent 6 bytes.
Sent 10 bytes.
Sent 7 bytes.
Sent 1 bytes.
```

In parallel we ran a packet capture on loopback filtered to the server's port, to see how these four writes actually travelled over TCP rather than trusting the application's own logging.

```
$ tcpdump -i lo0 -n port 5000
```

See Evidence File – Experiment 3.

4.2 Answer

Question: Does the Exchange Server receive the application-level message as one complete unit, or can the message be received in multiple pieces? Explain what this demonstrates about the relationship between TCP and application-level message boundaries.

The capture shows the client's payload arriving as several distinct **PSH**, **ACK** segments, not one:

```
...19064 > ...5000: Flags [P.], seq 1:7, length 6 (first write: 6 bytes)
...19064 > ...5000: Flags [P.], seq 17:24, length 7 (third write: 7 bytes)
...19064 > ...5000: Flags [P.], seq 24:25, length 1 (fourth write: 1 byte)
```

each segment's length matching one of the client's four **send()** calls (the 10-byte write shows up as its own segment too). These are separate TCP segments with different sequence-number ranges – not one packet with the full 24 bytes – which is exactly what the kernel hands back to the server across separate **recv()** calls, since **recv()** returns whatever's currently in the receive buffer rather than waiting for a "complete message."

This is the point Section 2.9 makes: TCP is just an ordered byte stream, with no concept of message boundaries between the sender's individual **send()** calls. Our protocol treats this as one logical newline-terminated line, but at the transport layer it's delivered however the OS decides to segment it – four pieces here, matching the four writes, but it could just as easily be fewer segments or a split at a totally different byte offset. This is exactly why **LineReader** (Section 1.2) accumulates bytes across as many **recv()** calls as needed and only treats a message as complete once a newline shows up, instead of assuming one **recv()** equals one message.

5 Experiment 4 — One Client Should Not Stall the Others

```
root@freebsd-a2:~ # cd socket-exchange-a2/
root@freebsd-a2:~/socket-exchange-a2 # python3 experiment.py 4
=== Experiment 4: One Client Should Not Stall the Others ===
```

```
Started Exchange Server (PID 4701).
Exchange Server listening on 127.0.0.1:5000
Exchange Server is listening.
```

```
Client 1: connected from 127.0.0.1:22525
```

```
Client 1 will send an incomplete application message.
Client 1 will now remain silent.
```

```
Client connected: ('127.0.0.1', 56694)
Client connected: ('127.0.0.1', 22525)
Client 2: connected from 127.0.0.1:39517
Client 2 will send a complete application message.
Client connected: ('127.0.0.1', 39517)
```

```
Client 2 response: 'OK'
Elapsed time: 0.004 seconds
```

Investigate the behavior of both TCP connections. Press Ctrl-C when you are finished.

Experiment finished.

```
Stopping Exchange Server...
root@freebsd-a2:~/socket-ex
```

[illegible]

```
root@freebsd-a2:~# cd socket-exchange-a2/
root@freebsd-a2:~/socket-exchange-a2# socskstat -4 -c
USER COMMAND PID FD PROTO LOCAL ADDRESS FOREIGN ADDRESS
root python3.12 4701 4 tcp4 127.0.0.1:5000 127.0.0.1:22525
root python3.12 4701 5 tcp4 127.0.0.1:5000 127.0.0.1:39517
root python3.12 4700 3 tcp4 127.0.0.1:22525 127.0.0.1:5000
root python3.12 4700 4 tcp4 127.0.0.1:39517 127.0.0.1:5000
root sshd-session 4527 7 tcp4 192.168.64.4:22 192.168.64.1:50723
root sshd-session 4551 4 tcp4 192.168.64.4:22 192.168.64.1:50723
root sshd-session 4525 7 tcp4 192.168.64.4:22 192.168.64.1:50717
root sshd-session 4522 7 tcp4 192.168.64.4:22 192.168.64.1:50717
root sshd-session 4519 7 tcp4 192.168.64.4:22 192.168.64.1:50713
root sshd-session 4516 7 tcp4 192.168.64.4:22 192.168.64.1:50713
root sshd-session 3847 7 tcp4 192.168.64.4:22 192.168.64.1:64138
root sshd-session 3839 7 tcp4 192.168.64.4:22 192.168.64.1:64138
root@freebsd-a2:~/socket-exchange-a2# you have mail
root@freebsd-a2:~/socket-exchange-a2# []
```

5.1 Investigation Approach

`experiment.py` 4 starts the server, connects Client 1, then shortly after connects Client 2 to try to communicate while Client 1 sits idle. Since this is really a timing question – can Client 2 be serviced promptly despite Client 1 being present? – we captured raw packets on loopback to see, to the millisecond, exactly when each connection’s handshake and traffic happened.

```
$ tcpdump -i lo0 -n port 5000
```

```
$ python3 experiment.py 4
```

See Evidence File – Experiment 4.

5.2 Answer

Question: When Client 1 remains connected but sends no data, can the Exchange Server still accept and service Client 2? Use your observations to identify the server operation that determines the answer.

Ignoring a couple of very short-lived probe connections at the start (ports 63528 and 56694, each reset within a fraction of a millisecond – likely the harness checking the port before the real experiment, same as Experiment 3), the two connections that matter are:

- **Client 1** – port 22525: handshake completes at 03:15:32.550581; sends a 20-byte message at 03:15:32.550658, ACK'd at 03:15:32.588334. Nothing else crosses this connection until it initiates the closing FIN at 03:16:41.959253 – idle for roughly **69 seconds**.
- **Client 2** – port 39517: connects almost two seconds later, handshake at 03:15:34.552754. Sends its own 20-byte message at 03:15:34.553197, gets a 3-byte reply at 03:15:34.557024 – a round trip of about **3.8 ms**.

So yes: the server can still accept and promptly service Client 2 while Client 1 sits idle. Client 2's request was answered in under 4 ms, with no delay from Client 1's silent open connection. Both connections closed together in an ordinary FIN/ACK exchange once the experiment ended.

The operation responsible is `accept()` plus our thread-per-connection design. The main thread only ever loops on `accept()`; the moment Client 1's connection is accepted, its socket goes to a dedicated worker thread, which blocks on its own `recv()` waiting for data that never comes. Since that block happens on Client 1's thread and not the main one, the main thread immediately calls `accept()` again for Client 2 and spawns a second, independent thread. If we'd instead handled clients sequentially on one thread – blocking on `recv()` for Client 1 before calling `accept()` again – Client 2 would have had to wait for Client 1 to send something, which never happens here.

Experiment 5(drushya)

13

14

[illegible][illegible]

6.1 Investigation Approach

`experiment.py` 5 opens five simultaneous connections, has Clients 1, 3, and 5 each send one message while Clients 2 and 4 stay idle, then holds all five open for an extended window. We looked at this two ways: a long-running `sockstat -4 -c poll` (once per second) to confirm what stayed connected, and a `tcpdump` capture to see which connections actually carried data.

```
$ python3 experiment.py 5
$ while true; do
    echo "=== $(date '+%H:%M:%S') ===" >> /tmp/exp5_log.txt
    sockstat -4 -c >> /tmp/exp5_log.txt
    sleep 1
done
$ tcpdump -i lo0 -n port 5000
```

See Evidence File – Experiment 5.

6.2 Answer

Question: At a particular point during the experiment, which client connections are actually ready for the server to service, and what evidence from the running system allows you to determine this?

Ignoring two short-lived probe connections reset right before the real traffic starts (same pattern as Experiments 3 and 4), `tcpdump` shows all five connections complete their handshake within the same millisecond (14:45:08.505124–.505603, ports 40868, 48261, 49520, 25283, 52152). Only three of the five ever carry a data-bearing segment afterward:

```
14:45:08.505748 40868 > 5000: [P.] seq 1:16, length 15 (Client 1 sends)
14:45:08.505919 5000 > 40868: [P.] seq 1:4, length 3 (server replies)
14:45:09.029234 49520 > 5000: [P.] seq 1:16, length 15 (Client 3 sends)
14:45:09.029665 5000 > 49520: [P.] seq 1:4, length 3 (server replies)
14:45:09.078887 52152 > 5000: [P.] seq 1:16, length 15 (Client 5 sends)
14:45:09.571248 5000 > 52152: [P.] seq 1:4, length 3 (server replies)
```

Ports 48261 (Client 2) and 25283 (Client 4) show no data-bearing packets at all between their handshake and teardown roughly 17 seconds later. This matches the application log (“Clients 1, 3, and 5 will send application messages. Clients 2 and 4 will remain idle.”) and tells us, at the packet level, exactly which sockets were readable: an I/O-multiplexing call like `select()/poll()/kqueue()` would only wake for Clients 1, 3, 5 (one at a time, roughly half a second apart), never for 2 and 4.

“Not ready” doesn’t mean “not there”: our `sockstat -4 -c poll` (once per second, 14:44:04–14:44:52, roughly 48 samples) shows all five connections – including the idle ones – listed as connected in every sample, no drops. The server keeps all five sockets open and monitorable the whole time regardless of whether they have anything to service, and only closes them together (synchronized FIN/ACK visible from 14:45:25.103445) once the experiment ends.

7 Experiment 6 — FIN vs. RST: Orderly and Abrupt Connection Termination

```
root@freeds-a2:~/socket-exchange-a2 # python3 experiment.py 6
== Experiment 6: FIN vs. RST: Orderly and Abrupt Connection Termination ==

Started Exchange Server (PID 5456).
Exchange Server listening on 127.0.0.1:5000
Exchange Server is listening.

Part A: orderly connection termination (FIN).
Orderly client: connected from 127.0.0.1:52116
The client will shut down its sending direction.
Investigate the TCP traffic and connection state.
Client connected: ('127.0.0.1', 45705)
Client connected: ('127.0.0.1', 52116)

Part B: abortive connection termination (RST).
Abrupt client: connected from 127.0.0.1:51257
The client will now close abortively.
Investigate the TCP traffic and connection state.

The Exchange Server will remain running for 15 seconds.
Client connected: ('127.0.0.1', 51257)

Experiment finished.
Stopping Exchange Server...
root@freeds-a2:~/socket-exchange-a2 #
```

[illegible]

```
root@freebsd-a2:~# cd socket-exchange-a2/
root@freebsd-a2:~/socket-exchange-a2# ssokstat -t -c
USER COMMAND      PID FD PROTO LOCAL ADDRESS          FOREIGN ADDRESS
root python3.12 5455 3 tcp4 127.0.0.1:52116       127.0.0.1:5000
root sshd-ssessi 5451 7 tcp4 192.168.64.4:122     192.168.64.1:53007
root sshd-ssessi 5448 7 tcp4 192.168.64.4:122     192.168.64.1:53007
root sshd-ssessi 5445 7 tcp4 192.168.64.4:122     192.168.64.1:53002
root sshd-ssessi 5442 7 tcp4 192.168.64.4:122     192.168.64.1:53002
root sshd-ssessi 5439 7 tcp4 192.168.64.4:122     192.168.64.1:52999
root sshd-ssessi 5436 7 tcp4 192.168.64.4:122     192.168.64.1:52999
root sshd-ssessi 5382 7 tcp4 192.168.64.4:122     192.168.64.1:64130
root sshd-ssessi 3839 7 tcp4 192.168.64.4:122     192.168.64.1:64130
root@freebsd-a2:~/socket-exchange-a2#
```

7.1 Investigation Approach

`experiment.py` 6 exercises both kinds of termination in one run: Part A connects an "orderly" client (port 52116) that half-closes its send side, Part B connects a separate "abrupt" client (port 51257) that tears the connection down abortively. We captured both parts in one continuous `tcpdump` session, and checked `sockstat -4 -c` to see what remained once Part A's close went through.

```
$ tcpdump -i lo0 -n port 5000
$ python3 experiment.py 6
$ sockstat -4 -c
```

See Evidence File – Experiment 6.

7.2 Answer

Question: How does an abrupt client termination differ from the orderly shutdown? Identify the TCP event observed on the network and the corresponding behavior of the Exchange Server’s socket.

Part A – orderly shutdown (port 52116). After the handshake, a clean, symmetric four-packet close:

```
04:16:57.806809 52116 > 5000: [F.] seq 1, ack 1 (client's FIN -- half-close)
04:16:57.807067 5000 > 52116: [.] ack 2 (server ACKs the FIN)
04:16:57.807596 5000 > 52116: [F.] seq 1, ack 2 (server's own FIN)
04:16:57.807652 52116 > 5000: [.] ack 2 (client ACKs server's FIN)
```

The client shuts down only its sending side, which shows up as an ordinary FIN. Since `read_line()` treats a `recv()` returning `b""` as a clean disconnect, the server’s handler exits and its `finally` block closes the socket almost immediately – about 0.8 ms after the client’s FIN – which is why the server’s own FIN follows so fast, completing a fully acknowledged close on both sides. A `sockstat` snapshot right after confirms this: only the client’s socket (PID 5455) still references port 52116; the server side is already gone.

Part B – abrupt termination (port 51257). Right after the handshake completes, the very next and last packet on this connection is a bare reset:

```
04:17:07.811787 51257 > 5000: [.] ack 1 (handshake completes, ESTABLISHED)
04:17:07.812875 51257 > 5000: [R.] seq 1, ack 1 (client sends RST)
```

No FIN, and unlike Part A, no ACK or FIN from the server afterward – an RST just isn’t acknowledged, and it’s the last packet the capture ever sees. On the server side, the same `recv()` that cleanly returned `b""` in Part A instead raises a `ConnectionResetError` here. We catch this and treat it the same as a clean disconnect for cleanup purposes, so `finally` still runs and the client gets removed from shared state – but the server learns about it through a failed-read exception rather than a graceful EOF, and it never gets to send a symmetric FIN, since the peer’s socket resources were already gone before any reply could go out.

In short: an orderly close is a negotiated four-packet FIN/ACK exchange where the server can still cleanly close and get that close acknowledged; an abrupt close is a single, unilateral RST with no acknowledgment and no further communication possible – the server only finds out via an error on its next read. The FIN→ACK→FIN→ACK sequence in Part A is the same one we inferred indirectly through state polling in Experiment 2, just seen directly here at the packet level.

8 Experiment 7 — Backpressure and the Slow Receiver

[illegible]

```

10      9      0      127.0.0.1.255      0.0.0.0      0.0.0.0      0.0.0.0
11
12      ...
13
14      ...
15
16      ...
17
18      ...
19
20      ...
21
22      ...
23
24      ...
25
26      ...
27
28      ...
29
30      ...
31
32      ...
33
34      ...
35
36      ...
37
38      ...
39
40      ...
41
42      ...
43
44      ...
45
46      ...
47
48      ...
49
50      ...
51
52      ...
53
54      ...
55
56      ...
57
58      ...
59
60      ...
61
62      ...
63
64      ...
65
66      ...
67
68      ...
69
70      ...
71
72      ...
73
74      ...
75
76      ...
77
78      ...
79
80      ...
81
82      ...
83
84      ...
85
86      ...
87
88      ...
89
90      ...
91
92      ...
93
94      ...
95
96      ...
97
98      ...
99
100     ...
101
102     ...
103
104     ...
105
106     ...
107
108     ...
109
110     ...
111
112     ...
113
114     ...
115
116     ...
117
118     ...
119
120     ...
121
122     ...
123
124     ...
125
126     ...
127
128     ...
129
130     ...
131
132     ...
133
134     ...
135
136     ...
137
138     ...
139
140     ...
141
142     ...
143
144     ...
145
146     ...
147
148     ...
149
150     ...
151
152     ...
153
154     ...
155
156     ...
157
158     ...
159
160     ...
161
162     ...
163
164     ...
165
166     ...
167
168     ...
169
170     ...
171
172     ...
173
174     ...
175
176     ...
177
178     ...
179
180     ...
181
182     ...
183
184     ...
185
186     ...
187
188     ...
189
190     ...
191
192     ...
193
194     ...
195
196     ...
197
198     ...
199
200     ...
201
202     ...
203
204     ...
205
206     ...
207
208     ...
209
210     ...
211
212     ...
213
214     ...
215
216     ...
217
218     ...
219
220     ...
221
222     ...
223
224     ...
225
226     ...
227
228     ...
229
230     ...
231
232     ...
233
234     ...
235
236     ...
237
238     ...
239
240     ...
241
242     ...
243
244     ...
245
246     ...
247
248     ...
249
250     ...
251
252     ...
253
254     ...
255
256     ...
257
258     ...
259
260     ...
261
262     ...
263
264     ...
265
266     ...
267
268     ...
269
270     ...
271
272     ...
273
274     ...
275
276     ...
277
278     ...
279
280     ...
281
282     ...
283
284     ...
285
286     ...
287
288     ...
289
290     ...
291
292     ...
293
294     ...
295
296     ...
297
298     ...
299
300     ...
301
302     ...
303
304     ...
305
306     ...
307
308     ...
309
310     ...
311
312     ...
313
314     ...
315
316     ...
317
318     ...
319
320     ...
321
322     ...
323
324     ...
325
326     ...
327
328     ...
329
330     ...
331
332     ...
333
334     ...
335
336     ...
337
338     ...
339
340     ...
341
342     ...
343
344     ...
345
346     ...
347
348     ...
349
350     ...
351
352     ...
353
354     ...
355
356     ...
357
358     ...
359
360     ...
361
362     ...
363
364     ...
365
366     ...
367
368     ...
369
370     ...
371
372     ...
373
374     ...
375
376     ...
377
378     ...
379
380     ...
381
382     ...
383
384     ...
385
386     ...
387
388     ...
389
390     ...
391
392     ...
393
394     ...
395
396     ...
397
398     ...
399
400     ...
401
402     ...
403
404     ...
405
406     ...
407
408     ...
409
410     ...
411
412     ...
413
414     ...
415
416     ...
417
418     ...
419
420     ...
421
422     ...
423
424     ...
425
426     ...
427
428     ...
429
430     ...
431
432     ...
433
434     ...
435
436     ...
437
438     ...
439
440     ...
441
442     ...
443
444     ...
445
446     ...
447
448     ...
449
450     ...
451
452     ...
453
454     ...
455
456     ...
457
458     ...
459
460     ...
461
462     ...
463
464     ...
465
466     ...
467
468     ...
469
470     ...
471
472     ...
473
474     ...
475
476     ...
477
478     ...
479
480     ...
481
482     ...
483
484     ...
485
486     ...
487
488     ...
489
490     ...
491
492     ...
493
494     ...
495
496     ...
497
498     ...
499
500     ...
501
502     ...
503
504     ...
505
506     ...
507
508     ...
509
510     ...
511
512     ...
513
514     ...
515
516     ...
517
518     ...
519
520     ...
521
522     ...
523
524     ...
525
526     ...
527
528     ...
529
530     ...
531
532     ...
533
534     ...
535
536     ...
537
538     ...
539
540     ...
541
542     ...
543
544     ...
545
546     ...
547
548     ...
549
550     ...
551
552     ...
553
554     ...
555
556     ...
557
558     ...
559
560     ...
561
562     ...
563
564     ...
565
566     ...
567
568     ...
569
570     ...
571
572     ...
573
574     ...
575
576     ...
577
578     ...
579
580     ...
581
582     ...
583
584     ...
585
586     ...
587
588     ...
589
590     ...
591
592     ...
593
594     ...
595
596     ...
597
598     ...
599
600     ...
601
602     ...
603
604     ...
605
606     ...
607
608     ...
609
610     ...
611
612     ...
613
614     ...
615
616     ...
617
618     ...
619
620     ...
621
622     ...
623
624     ...
625
626     ...
627
628     ...
629
630     ...
631
632     ...
633
634     ...
635
636     ...
637
638     ...
639
640     ...
641
642     ...
643
644     ...
645
646     ...
647
648     ...
649
650     ...
651
652     ...
653
654     ...
655
656     ...
657
658     ...
659
660     ...
661
662     ...
663
664     ...
665
666     ...
667
668     ...
669
670     ...
671
672     ...
673
674     ...
675
676     ...
677
678     ...
679
680     ...
681
682     ...
683
684     ...
685
686     ...
687
688     ...
689
690     ...
691
692     ...
693
694     ...
695
696     ...
697
698     ...
699
700     ...
701
702     ...
703
704     ...
705
706     ...
707
708     ...
709
710     ...
711
712     ...
713
714     ...
715
716     ...
717
718     ...
719
720     ...
721
722     ...
723
724     ...
725
726     ...
727
728     ...
729
730     ...
731
732     ...
733
734     ...
735
736     ...
737
738     ...
739
740     ...
741
742     ...
743
744     ...
745
746     ...
747
748     ...
749
750     ...
751
752     ...
753
754     ...
755
756     ...
757
758     ...
759
760     ...
761
762     ...
763
764     ...
765
766     ...
767
768     ...
769
770     ...
771
772     ...
773
774     ...
775
776     ...
777
778     ...
779
780     ...
781
782     ...
783
784     ...
785
786     ...
787
788     ...
789
790     ...
791
792     ...
793
794     ...
795
796     ...
797
798     ...
799
800     ...
801
802     ...
803
804     ...
805
806     ...
807
808     ...
809
810     ...
811
812     ...
813
814     ...
815
816     ...
817
818     ...
819
820     ...
821
822     ...
823
824     ...
825
826     ...
827
828     ...
829
830     ...
831
832     ...
833
834     ...
835
836     ...
8
```

8.1 Investigation Approach

`experiment.py` 7 sets up two Trader Clients continuously trading JNST against each other (a steady stream of `TRADE` broadcasts) plus two Market-Data Clients subscribed to JNST – one (“Normal”) reading updates promptly, one (“Slow”) that deliberately stops reading. We polled `netstat -an | grep 5000` once per second for several minutes to track Recv-Q/Send-Q on every connection over time.

```
$ python3 experiment.py 7
$ while true; do
    echo "=== $(date '+%H:%M:%S') ===" >> /tmp/exp7_log.txt
    netstat -an | grep 5000 >> /tmp/exp7_log.txt
    sleep 1
done
```

See Evidence File – Experiment 7.

8.2 Answer

Question: How does the Exchange Server’s TCP connection to the slow client behave as the client stops reading, and what evidence shows whether this eventually affects the server’s ability to communicate with other clients?

The slow client’s Recv-Q – bytes delivered by TCP but not yet consumed – grows without pause:

```
13:43:08 Recv-Q = 17
13:43:09 Recv-Q = 476
13:43:10 Recv-Q = 969
13:43:11 Recv-Q = 1411
...
13:44:30 Recv-Q = 34697
13:44:34 Recv-Q = 36431
...
13:46:29 -- 13:46:42 Recv-Q = 85000 (unchanged for the rest of the run)
```

It climbs for a couple of minutes then flatlines at exactly 85000 bytes – the receive buffer is full, its advertised TCP window drops to zero. Correspondingly, the server’s Send-Q for that same connection climbs briefly (17 → 34 → 72 bytes) then settles back to 0 once the client’s window closes – the server’s own send buffer for this socket is full too, and further `send()` calls for this client would block until it starts reading again, which it never does.

Meanwhile the Normal Market-Data Client and both Trader connections stay small and flat throughout – tens of bytes, never climbing into the thousands. That’s the clearest evidence the slow receiver’s backlog stayed isolated to its own socket rather than spreading.

One subtlety worth flagging: `broadcast_trade()` sends each `TRADE` update to every subscriber, including the slow one, from inside the trader thread currently holding `state_lock`. Once the slow client’s send buffer is full (from roughly 13:46:29 on), that `send_line()` call would block on that thread until the client drains its buffer – and since `state_lock` is held, any other operation needing that lock has to wait behind it. Our `netstat` evidence alone can’t confirm whether trades kept being generated after this point, so we flag this as an implementation-level risk rather than something we can prove happened. What the evidence does conclusively show is that the slow client’s connection backed up end-to-end while the other three stayed consistently small.

9 Experiment 8 — Unexpected Client Disconnection

```
== 04:33:54 ==
```

USER	COMMAND	PID	FD	PROTO	LOCAL ADDRESS	FOREIGN ADDRESS
root	python3.12	7830	3	tcp4	127.0.0.1:20545	127.0.0.1:5000
root	python3.12	7828	4	tcp4	127.0.0.1:5000	127.0.0.1:35912
root	python3.12	7828	5	tcp4	127.0.0.1:5000	127.0.0.1:13796
root	python3.12	7828	6	tcp4	127.0.0.1:5000	127.0.0.1:63257
root	python3.12	7828	7	tcp4	127.0.0.1:5000	127.0.0.1:20545
root	python3.12	7827	3	tcp4	127.0.0.1:35912	127.0.0.1:5000
root	python3.12	7827	4	tcp4	127.0.0.1:13796	127.0.0.1:5000
root	python3.12	7827	5	tcp4	127.0.0.1:63257	127.0.0.1:5000
root	sshd-session	7759	7	tcp4	192.168.64.4:22	192.168.64.1:53524
root	sshd-session	7756	7	tcp4	192.168.64.4:22	192.168.64.1:53524
root	sshd-session	7753	7	tcp4	192.168.64.4:22	192.168.64.1:53520
root	sshd-session	7750	7	tcp4	192.168.64.4:22	192.168.64.1:53520
root	sshd-session	7747	7	tcp4	192.168.64.4:22	192.168.64.1:53516
root	sshd-session	7744	7	tcp4	192.168.64.4:22	192.168.64.1:53516
root	sshd-session	3842	7	tcp4	192.168.64.4:22	192.168.64.1:64130
root	sshd-session	3839	7	tcp4	192.168.64.4:22	192.168.64.1:64130

```
== 04:33:55 ==
```

USER	COMMAND	PID	FD	PROTO	LOCAL ADDRESS	FOREIGN ADDRESS
root	python3.12	7830	3	tcp4	127.0.0.1:20545	127.0.0.1:5000
root	python3.12	7828	4	tcp4	127.0.0.1:5000	127.0.0.1:35912
root	python3.12	7828	5	tcp4	127.0.0.1:5000	127.0.0.1:13796

```
== 04:33:56 ==
```

USER	COMMAND	PID	FD	PROTO	LOCAL ADDRESS	FOREIGN ADDRESS
root	python3.12	7830	3	tcp4	127.0.0.1:20545	127.0.0.1:5000
root	python3.12	7828	4	tcp4	127.0.0.1:5000	127.0.0.1:35912
root	python3.12	7828	5	tcp4	127.0.0.1:5000	127.0.0.1:13796
root	python3.12	7828	6	tcp4	127.0.0.1:5000	127.0.0.1:63257
root	python3.12	7828	7	tcp4	127.0.0.1:5000	127.0.0.1:20545
root	python3.12	7827	3	tcp4	127.0.0.1:35912	127.0.0.1:5000
root	python3.12	7827	4	tcp4	127.0.0.1:13796	127.0.0.1:5000
root	python3.12	7827	5	tcp4	127.0.0.1:63257	127.0.0.1:5000
root	sshd-session	7759	7	tcp4	192.168.64.4:22	192.168.64.1:53524
root	sshd-session	7756	7	tcp4	192.168.64.4:22	192.168.64.1:53524
root	sshd-session	7753	7	tcp4	192.168.64.4:22	192.168.64.1:53520
root	sshd-session	7750	7	tcp4	192.168.64.4:22	192.168.64.1:53520
root	sshd-session	7747	7	tcp4	192.168.64.4:22	192.168.64.1:53516
root	sshd-session	7744	7	tcp4	192.168.64.4:22	192.168.64.1:53516
root	sshd-session	3842	7	tcp4	192.168.64.4:22	192.168.64.1:64130
root	sshd-session	3839	7	tcp4	192.168.64.4:22	192.168.64.1:64130

```
== 04:33:57 ==
```

USER	COMMAND	PID	FD	PROTO	LOCAL ADDRESS	FOREIGN ADDRESS
root	python3.12	7828	4	tcp4	127.0.0.1:5000	127.0.0.1:35912
root	python3.12	7828	5	tcp4	127.0.0.1:5000	127.0.0.1:13796
root	python3.12	7828	6	tcp4	127.0.0.1:5000	127.0.0.1:63257
root	python3.12	7827	3	tcp4	127.0.0.1:35912	127.0.0.1:5000
root	python3.12	7827	4	tcp4	127.0.0.1:13796	127.0.0.1:5000
root	python3.12	7827	5	tcp4	127.0.0.1:63257	127.0.0.1:5000
root	sshd-session	7759	7	tcp4	192.168.64.4:22	192.168.64.1:53524
root	sshd-session	7756	7	tcp4	192.168.64.4:22	192.168.64.1:53524
root	sshd-session	7753	7	tcp4	192.168.64.4:22	192.168.64.1:53520
root	sshd-session	7750	7	tcp4	192.168.64.4:22	192.168.64.1:53520
root	sshd-session	7747	7	tcp4	192.168.64.4:22	192.168.64.1:53516
root	sshd-session	7744	7	tcp4	192.168.64.4:22	192.168.64.1:53516
root	sshd-session	3842	7	tcp4	192.168.64.4:22	192.168.64.1:64130
root	sshd-session	3839	7	tcp4	192.168.64.4:22	192.168.64.1:64130

```
== 04:33:58 ==
```

USER	COMMAND	PID	FD	PROTO	LOCAL ADDRESS	FOREIGN ADDRESS
root	python3.12	7828	4	tcp4	127.0.0.1:5000	127.0.0.1:35912
root	python3.12	7828	5	tcp4	127.0.0.1:5000	127.0.0.1:13796
root	python3.12	7828	6	tcp4	127.0.0.1:5000	127.0.0.1:63257
root	python3.12	7827	3	tcp4	127.0.0.1:35912	127.0.0.1:5000
root	python3.12	7827	4	tcp4	127.0.0.1:13796	127.0.0.1:5000
root	python3.12	7827	5	tcp4	127.0.0.1:63257	127.0.0.1:5000
root	sshd-session	7759	7	tcp4	192.168.64.4:22	192.168.64.1:53524

9.1 Investigation Approach

experiment.py 8 connects a "Surviving" Market-Data Client (port 35912) alongside a second Market-Data Client (port 20545, run as its own independent OS process) while a Buyer and Seller

trade JNST continuously, generating trade broadcasts to both subscribers. Partway through, the harness kills the second client's process to simulate a crash, then keeps generating trades to see if the remaining connections still get serviced. We polled `sockstat -4 -c` once per second to pin down exactly when each side's socket disappeared.

```
$ python3 experiment.py 8
$ while true; do
    echo "=== $(date '+%H:%M:%S') ===" >> /tmp/exp8_log.txt
    sockstat -4 -c >> /tmp/exp8_log.txt
    sleep 1
done
```

See Evidence File – Experiment 8.

9.2 Answer

Question: When a client disappears unexpectedly while the Exchange Server is communicating with it, what happens to their TCP connection, and what evidence allows you to determine how the server detects the failure?

The moment the client's process is killed, the server logs "Market-data client ('127.0.0.1', 20545) disconnected." Our `sockstat` polling pins this to within a couple of seconds: at 04:33:54–04:33:55, both the killed client's socket (PID 7830) and the server's accepted socket for it (PID 7828, FD 7) are still listed. By the next sample, 04:33:56, both are gone – PID 7830 doesn't appear at all (process is dead), and FD 7 is gone from the server's list too. This matches our implementation: whichever way the OS reports the disconnection (clean EOF or reset), `read_line()` treats it as a disconnect, `finally` runs, and the client is removed from `subscribers` – all within about a second of the kill.

The three surviving connections – the other Market-Data Client, Buyer, Seller – stay listed, unchanged, in every one of the roughly 45 further samples through 04:34:37, and the log confirms trades kept being generated and delivered well after the disconnection. That's direct evidence one client's failure had no effect on the server's ability to service the others – consistent with thread-per-connection, where each socket failure is handled entirely within that client's own thread and `finally` block.

One caveat: unlike Experiment 6, we didn't run `tcpdump` at the exact moment of the kill here, so we can't say from packet-level evidence whether cleanup produced a FIN or an RST on the wire. Our conclusion rests on the socket-table evidence and the server's own log message rather than a packet capture – but either way, this is an unexpected disconnection (the client never sent QUIT), and our cleanup path handles both possible network-level events identically.

10 Bonus — Connection Scalability and I/O Design

[illegible]

```

root@freedsb_a2:~/socket-exchange-a2 # netstat -an | grep 5000 | grep ESTABLISHED | wc -l
100000
root@freedsb_a2:~/socket-exchange-a2 # ps -o pid,rss,vsz,Ncpu -p 1436
PID      RSS          VSZ      NCPU
1436 2929912 8326668    0.0
root@freedsb_a2:~/socket-exchange-a2 # procstat -f 1436 | wc -l
30010
root@freedsb_a2:~/socket-exchange-a2 # sysctl kern.openfiles
kern.openfiles: 68113
root@freedsb_a2:~/socket-exchange-a2 # netstat -an | grep 5000 | grep ESTABLISHED | wc -l
80000
root@freedsb_a2:~/socket-exchange-a2 # ps -o pid,rss,vsz,Ncpu -p 1436
PID      RSS          VSZ      NCPU
1436    0 11346412    0.0
root@freedsb_a2:~/socket-exchange-a2 # procstat -f 1436 | wc -l
40010
root@freedsb_a2:~/socket-exchange-a2 # sysctl kern.openfiles
kern.openfiles: 80113
root@freedsb_a2:~/socket-exchange-a2 # █

```

```

root@freedsb_a2:~/socket-exchange-a2 # netstat -an | grep 5000 | grep ESTABLISHED | wc -l
100000
root@freedsb_a2:~/socket-exchange-a2 # ps -o pid,rss,vsz,Ncpu -p 1436
PID      RSS          VSZ      NCPU
1436 3178416 13992396    0.0
root@freedsb_a2:~/socket-exchange-a2 # procstat -f 1436 | wc -l
50010
root@freedsb_a2:~/socket-exchange-a2 # sysctl kern.openfiles
kern.openfiles: 100113
root@freedsb_a2:~/socket-exchange-a2 # netstat -an | head -5
777/1478/2055 mbufs in use (current/cache/total)
0/762/762/552488 mbuf clusters in use (current/cache/total/max)
0/8088 mbuf/clusters out of packet secondary zone in use (current/cache)
328/662/982/126728 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37849 0k jumbo clusters in use (current/cache/total/max)
root@freedsb_a2:~/socket-exchange-a2 # top -b -n 1 | head -10
last pid: 1610; load averages: 3.26, 66.26, 42.69 up 0@0:59:02 11:27:15
36 processes: 1 running, 35 sleeping
CPU: 1.7% user, 0.0% nice, 6.4% system, 0.6% interrupt, 92.3% idle
Mem: 166M Active, 158M Inact, 343M Laundry, 166M Wired, 205K Buf, 22M Free
ARC: 35M Total, 11M MFU, 22M MRU, 267K Header, 1420K Other
10M Compressed, 52M Uncompressed, 2.95:1 Ratio
Swap: 284M Total, 1819M Used, 229M Free, 98% Inuse

PID USERNAME THR PRI NICE SIZE RES STATE C TIME WCPU COMMAND
1463 root 1 0 0 22M 300K select 1 0:00 0.10% sshd-session
root@freedsb_a2:~/socket-exchange-a2 # █

```

Idle Conns	Mem (RSS)	Mem (Virtual)	CPU (%)	Server FDs	System FDs
10,000	1,017 MB	2,824 MB	0.0	10,010	20,103
20,000	1,856 MB	5,516 MB	0.0	20,010	40,113
30,000	2,861 MB	8,132 MB	0.0	30,010	60,113
40,000	0 MB*	11,081 MB	0.0	40,010	80,113
50,000	3,104 MB	13,664 MB	0.0	50,010	100,113
60,000		<i>Not achieved – see Analysis Q1</i>			
70,000		<i>Not achieved – see Analysis Q1</i>			

Table 1: Resource usage vs. number of idle connections. *At 40,000 connections `ps` reported `RSS = 0B`, which we take to be a measurement artifact of active swapping at this connection count (see Analysis, Q1–Q2) rather than a real reading – we independently confirmed via `procstat` and `netstat` that the process was alive and correctly holding all 40,000 connections. Virtual memory size (VSZ), which doesn’t depend on whether pages are currently resident or swapped, is used as our primary memory-scaling metric throughout this section.

File – Bonus – 10,000 / 40,000 / 50,000 idle connections.

10.3 Analysis

1. Was the server able to maintain all requested connections at each count? If not, at which count did it first fail?

The server held *all* requested connections cleanly at 10,000, 20,000, 30,000, 40,000, and 50,000 – every time, 100% of connection attempts succeeded (0 failures from our generator), and `netstat/procstat` confirmed the exact expected count of established connections and open FDs. At 50,000, though, our `top` snapshot showed the system was already in real trouble: 88% of our 2 GB swap was used, and the 5- and 15-minute load averages had spiked to 56.3 and 42.7 – extremely high for a machine that should be almost entirely idle. When we then tried to close this batch, the generator’s own `close()` loop hung, and shortly after, even a plain `netstat` command run in a separate terminal hung too. The VM stopped responding entirely and had to be force-restarted from the hypervisor. We didn’t attempt 60,000 or 70,000 after that: the 50,000 state was already at the edge of what the system could sustain, since pushing past it made the machine unusable before we could measure anything further. Our practical scalability ceiling is roughly **50,000 connections** on this VM (4 GB RAM, 2 GB swap).

2. What is the first significant bottleneck encountered, and what evidence from the table supports this (memory, CPU, file-descriptor limits, socket-buffer limits, or another OS resource)?

The bottleneck is **memory** – specifically, memory tied up in per-connection thread stacks. Two earlier, smaller limits had to be raised before memory became the real constraint: our server’s original `listen(20)` backlog was too small and caused connection attempts to fail with `ECONNRESET` once a few hundred arrived in quick succession, fixed by raising it to `listen(1024)` (plus the matching FreeBSD `kern.ipc.soacceptqueue` limit). We also found FreeBSD’s default `kern.threads.max_threads_per_proc` is exactly 1500, and since our server spawns one thread per connection, it reliably crashed with `RuntimeError: can’t start new thread` right around the 1,500th connection until we raised this to 100,000. Neither of these was the eventual bottleneck though – once both were fixed, connections scaled cleanly and CPU stayed flat at 0.0% the whole time (expected, since idle connections

do essentially no work). File descriptors weren't close to their limit either: at 50,000 connections we were using 100,113 of a 200,000 ceiling, under half. Instead, virtual memory grew almost perfectly linearly with connection count across the whole range: roughly **280 KB per idle connection** (2,824 MB at 10,000 connections, 13,664 MB at 50,000 – a near-constant per-connection cost across a 5× range). With only 4 GB RAM and 2 GB swap on this VM, that per-thread cost is what pushed the system into active swapping by 40,000 connections (23% swap used) and near-total exhaustion by 50,000 (88% swap used, 229 MB left) – right where the system stopped being able to service even basic shell commands.

3. How does your concurrency/I/O design (thread/process-per-connection vs. an I/O-multiplexing mechanism such as `poll()`/`kqueue()`) contribute to the observed bottleneck?

Our thread-per-connection design is the direct cause. Every connection, however idle, gets its own OS thread that lives for the connection's whole lifetime, and each thread needs the kernel to reserve a full stack plus scheduling structures whether or not that thread is actually doing anything. Since our idle connections just block on `recv()` forever, basically all of the ~280 KB per connection we measured is thread-stack overhead, not useful per-connection state – the actual TCP socket structures the kernel needs are a small fraction of that. This is exactly the trade-off we flagged in Section 1.1 when justifying thread-per-connection over `select()`/`poll()`/`kqueue()`: simple to write, but the cost scales with the number of connections rather than the number of connections actually doing something – which is the worst possible case for this idle-heavy test.

4. Would changing the I/O/concurrency mechanism help with the bottleneck identified? Explain why or why not.

Yes, a lot. A `kqueue()`-based design would use a small, constant number of threads – possibly just one – regardless of connection count. Instead of one thread blocking per connection, a single thread would register every socket with a `kqueue()` instance and only wake up when a specific one actually has data, exactly as we saw directly in Experiment 5: idle sockets never generate a readiness event, so they'd never need a thread of their own. Memory would then scale with the number of *active* connections and a tiny fixed amount of per-connection bookkeeping – a few hundred bytes, not ~280 KB. Given that our measured bottleneck (memory, almost entirely thread stacks) would basically disappear, we'd expect the same 4 GB VM to comfortably handle 70,000+ idle connections under an event-driven design – we still had plenty of CPU and FD headroom at 50,000 (0% CPU, under half our FD budget used), which strongly suggests memory was the only thing actually limiting us.

5. Quantify the trade-off: if you changed your implementation/configuration, what did the re-measured resource usage look like compared to the original design?

We didn't have time to fully reimplement the server with `kqueue()` for a direct side-by-side comparison at scale. What we do have is the measured, consistent ~280 KB-per-connection virtual-memory cost of the thread-per-connection design itself (Table 1), which held steady across every connection count we tested, 10,000 through 50,000. As a lighter-weight fix within the same design, reducing thread stack size via `threading.stack_size()` (which defaults to several hundred KB to a few MB on FreeBSD, well above the ~280 KB we measured) should cut this figure directly and proportionally, without needing a different concurrency model – though it wouldn't fix the underlying issue that we're still using one kernel-scheduled thread per connection, which `kqueue()` avoids entirely. Based on our ~280 KB/connection figure and the fact that a `kqueue()`-based design's per-connection cost would be two to three orders of magnitude smaller (a few hundred

bytes of bookkeeping vs. a full thread stack), we'd estimate the same 4 GB VM could sustain well over an order of magnitude more idle connections under an event-driven redesign than the ~50,000-connection ceiling we hit here.